An Introduction to the 3PL Language

Paul Dunn

May 2008

Open Report CMSE-2008-150

Contents

1 Preface

This 3PL tutorial gives an outline of the language and attempts to introduce some of its more unusual aspects. It does not attempt to document all the features of the language and the 3PL manual should be consulted for more detail. In the following the word **target** refers to the FPGA, i.e. the intended destination of the parallel pipelined code.

3PL is a language intended to allow parallel algorithms to be coded for implementation in an FPGA. A language which compiled directly into an FPGA configuration was considered insufficient on its own since it is often necessary to run an algorithm which in turn generates the target algorithm. For example an extreme case is a batcher sorting network where the batcher algorithm must be executed to generate the network topology. In simpler cases it is an advantage to parameterise the target algorithm which again implies some sort of preliminary processing. This could be accomplished by using a general purpose language such as C, Java or Python to pre-process the target algorithm, but it was decided that it was preferable to combine all of this in one language. Thus 3PL is actually a sequential language similar in most respects to C, but it has additional statement types that generate FPGA target code. This might have been accomplished by adding appropriate classes to Java or Python but it was felt that a new language tailored to the task would be preferable.

3PL was designed to meet several criteria -

- it must provide constructs within the FPGA such as variables, memory and combinatorial logic.
- it must provide execution threads which include sequential operations, parallel operations and looping and conditional control.
- it should provide a mechanism whereby systolic pipelines can be constructed which are data driven.
- the control of execution threads must co-exist with data-driven execution.
- it should be concise.

2 Variable modes

Variables in 3PL fall into 11 categories, referred to as **modes**. These are -

Immediate - an immediate variable only exists during execution of the 3PL interpreter. When a 3PL program is executed by the interpreter immediate mode variables are used in the same way as variables in languages such as C. Immediate mode variables do not generate FPGA code and are not represented in the FPGA configuration in any way. Immediate mode variables can be integer, boolean, floating point, string, array, struct, pointer, file or enumerated types. Immediate execution is discussed later.

A program containing only immediate mode variables will execute in a similar way to a C program and no FPGA code will be involved.

The remaining modes are all 'target' modes, that is they represent hardware devices in the FPGA.

Static - a static variable is a register in the FPGA. It can be integer, unsigned integer, fixed point, unsigned fixed point, floating point, boolean, array or struct types. All except the boolean type must have bit widths specified.

Queue - a variable which is a queue. If it is evaluated in a statement a datum will be popped. If it is assigned, a datum will be pushed. Otherwise it appears in statements and expressions in the same way as other variables. In any context a statement using a queue variable will wait to execute if the variable is not 'available', i.e. has a datum to pop or a free space in which to push a datum as appropriate.

Value - a value variable can represent any expression. The expression could be as simple as a single static or queue variable, in which case the value variable is a bit like a pointer.

Selectvalue - a variable which can select one of many inputs dynamically. In earlier FPGAs this would be implemented by 3-state buffers, but in later platforms the implementation uses combinatorial logic. These variables are used to construct buses internal to the FPGA.

Priority - a priority variable implements a priority encoder. It is combinatorial logic which appears as a boolean array where multiple simultaneous inputs are blocked except for that with the highest priority.

Input - these provide an interface for input pads.

Output - output variables drive 2-state or 3-state output pads.

Clock - clock variables provide clocks for the synchronous logic which 3PL produces. Connections between clock inputs, clock managers and clock buffers are also clock mode variables.

Rmemory - these create registered-output memory, implemented at the moment by block RAM.

Cmemory - these create combinatorial memory, implemented using CLB RAM.

3 Static mode variables

The most straightforward of the target modes is **static**. A static mode variable is equivalent to a normal variable in programming languages such as C or Java. It is implemented in the FPGA as a register using flip-flops. A static variable stores a value and therefore has state.

The following code shows the creation of a static variable and an assignment to it -

```
static(v, "int:8");
v <- 3;
```

The string "int:8" gives the type of the variable, which is an 8-bit integer.

The assignment operator for a static variable is `<-`. There are a number of different assignment operators for different variable modes, the intention being to make the modes immediately distinguishable on inspection of source code without the need in most cases to look for the statement creating the variable. Thus an assignment using `<-` immediately indicates that the variable on the left side is a static variable.

The assignment takes one clock cycle.

On assignment the widths of the left and right hand sides of the assignment need not be the same, however in that case padding or truncation will result. If the type is signed then padding will be by sign extension, otherwise zero padding is used.

3PL has the unusual characteristic that FPGA statements execute repeatedly, i.e. as if they were enclosed in infinite loops. The reason for this design decision is that FPGA statements are intended to operate on data streams and to do so with a high degree of parallelism. Thus every statement can potentially execute on every clock cycle and it is up to conditional code to control when statements execute. In a parallel data-driven system it seems more sensible to have execution on every clock cycle as the default with an overriding conditional mechanism to choose when to execute, rather than to enclose every statement in an explicit iteration loop.

In the example above the variable **v** will be assigned 3 on every clock cycle. This is not very useful, so we could use a conditional statement to exercise some control, e.g.

```
static(v, "int:8");
static(l, "log");
when (l)
    v <- 3;
```

Here we have introduced another static variable **l** which has a logical (boolean) type. When **l** is true, **v** is assigned 3. Now the **when** is executed repeatedly and if **l** is true the assignment is executed otherwise (since there is no **else** block) nothing is executed. Note that the **when** does not wait for **l** to be true, it simply evaluates **l** on every one of its continuous iterations and when true executes the code body, in this case a single assignment. To make the example more useful we could implement a controlled counter by the code -

```
static(v, "int:8");
static(l, "log");
when (l)
    v <- v + 1;
```

where **v** is incremented on clock cycles when **l** is true. 3PL allows the special case of addition or subtraction of one to/from a variable to be written using ++ and -- unary operators, e.g.

```
static(v, "int:8");
static(l, "log");
when (l)
    v++;
```

but note that these operators cannot be used on a static variable as part of an expression. We could make the counter bi-directional, e.g.

```
static(v, "int:8");
static({l, up}, "log");
when (l)
    when (up)
        v++;
    else
        v--;
```

where when **l** is true **v** will be incremented if **up** is true or decremented if **up** is false. As illustrated here many variable creation procedures allow lists of variables in braces where more than one variable of the same type is required.

The above example is quite correct, however it is preferable to avoid nested conditional statements where possible as this can lead to long path delays in the FPGA logic. We could recode the example as follows -

```
static(v, "int:8");
static({l, up}, "log");
when (l && up)
    v++;
when (l && !up)
    v--;
```

which achieves the same behaviour but reduces logic depth at the expense of logic width.

Static variables have default initial values which are 0 for numeric types and false for the logical type. A static variable can be given an initial value by means of the optional 3rd argument to procedure **static()**, e.g.

```
static(v, "int:8", 117);
```

will give static variable **v** an initial value of 117. If numeric the initial value can of course use hexadecimal (0x...), octal(0...) or binary (0b...) notation.

4 Input and output mode variables

We need to be able to get data into and out of the FPGA. For this purpose there are two variable modes, **input** and **output**.

An input mode variable represents signals coming into the FPGA via pads and input buffers. The following code latches input data into a static variable when an enable signal is high -

```
str(enable_pin, "D8");
var(din_pins, "[8]str", {"A1", "A2", "B1", "B2", "C1", "C2", "D1", "D2"});
input(d_in, "int:8", loc=din_pins);
input(enable, "log", loc=enable_pin);
static(d, type(d_in));
when (enable)
    d <- d_in;
```

Input mode variables are created by calling input() which has three or more arguments.

The first argument to input() is an identifier for the created variable.

The second argument to input() is the type, in this case one is an integer of 8 bits and the other is logical (boolean) and hence is implicitly one bit.

The third and following arguments are attributes for the input. This can include FPGA pins (required), a timing group identifier string, a flag to control the inserted input path delay, a flag to indicate that an IBUFG is to be used rather than an IBUF and finally flags for pullups or pulldowns. The attributes are passed in the form attribute=value.

In general arguments can be passed by position in the normal way or else by key=value where key is the name of the parameter for which the argument is intended (this is referred to in the 3PL manual as keymatch format). Keymatch format is useful where the parameters are many and the arguments are sparse. In 3PL arguments are optional and parameters can have default values specified for cases where no argument is passed to them.

In the code above the **loc** attribute must either be a string or an array or strings. These are used to generate FPGA pad location constraints. Note that array dimensions precede the array type.

A string immediate mode variable can be created by calling **str()**. The first argument is an identifier for the variable or else a list of identifiers in brackets. The second argument is optional and is the initial value of the string.

An array of strings is created by the **var()** procedure. This is the basic variable creation procedure for most modes of variable, procedures such as **static()**, being convenient equivalents. The code

```
static(l, "log");
str(s, "a_string");
```

is equivalent to

```
var(l, static, "log");
var(s, "str", "a_string");
```

If the second argument is not a mode identifier then it is interpreted as the variable type and the mode is assumed to be immediate. The var() procedure entry in the 3PL manual should be consulted for more details.

The input mode example above assigns the input mode variable **d_in** to the static variable **d** when the input mode variable **enable** is true (high).

Note the use of the function **type()** in the example - this simply returns the type of its argument, providing a simple way of creating a variable whose type is the same as another.

The above example would produce a fatal error in 3PL as the input variable **e**nable is used to control conditional logic but is driven by an external source and has no timing information with respect to the current clock, so there could potentially be timing violations. The usual way to handle this is to register the inputs. We can also instruct the place-and-route software to place the register flip-flops in an I/O block by setting an attribute -

```
str(enable_pin, "D8");
var(din_pins, "[8]str", {"A1", "A2", "B1", "B2", "C1", "C2", "D1", "D2"});
input(d_in, "int:8", loc=din_pins);
input(enable_in, "log", loc=enable_pin);
static(d, type(d_in), IOB=true);
static(enable, "log", IOB=true);

static(dd, type(d));

d <- d_in;
enable <- enable_in;

when (enable)
    dd <- d;
```

With the inputs d_in and enable_in now latched into flip-flops within I/O blocks we minimise the delay between input pad and flip-flop D pins. The clock used for this logic must bear an appropriate phase relationship with the external data input such that setup and hold constraints are met.

We could expand the example by connecting the static variable **dd** to output pins -

```
str(enable_pin, "D8");
var(din_pins, "[8]str", {"A1", "A2", "B1", "B2", "C1", "C2", "D1", "D2"});
var(dout_pins, "[8]str", {"E1", "E2", "F1", "F2", "G1", "G2", "H1", "H2"});
input(d_in, "int:8", loc=din_pins);
input(enable_in, "log", loc=enable_pin);
static(d, type(d_in), IOB=true);
static(enable, "log", IOB=true);
output(,, dd, loc=dout_pins);

static(dd, type(d));

d <- d_in;
enable <- enable_in;

when (enable)
    dd <- d;
```

The output() procedure has four or more arguments. The first is an identifier, but since in this case we have no need to refer to it we need no identifier so the argument is omitted. The second argument is the type, but the type can be inferred from the third argument so again the argument can be omitted. The third argument is optional and is the source to be used for the output (there are other ways of connecting an output mode variable to its source, in which case the third argument would be omitted but the first two would be required). The fourth and following arguments provide attributes as for the input() procedure. In this case they can include slew rate and drive current, the pin locations again being mandatory.

The **output()** statement connects the static variable **out** to the FPGA pads using output buffers. This example could be re-written to make the output assignment explicit -

```
str(enable_pin, "D8");
var(din_pins, "[8]str", {"A1", "A2", "B1", "B2", "C1", "C2", "D1", "D2"});
var(dout_pins, "[8]str", {"E1", "E2", "F1", "F2", "G1", "G2", "H1", "H2"});
input(d_in, "int:8", loc=din_pins);
input(enable_in, "log", loc=enable_pin);
static(d, type(d_in), IOB=true);
static(enable, "log", IOB=true);
static(dd, type(d));
output(out, type(dd),, loc=dout_pins);

d <- d_in;
enable <- enable_in;

when (enable)
    dd <- d;

out := dd;
```

note that the last assignment is immediate, not target. It is like assignment to a value variable and hence uses the same assignment operator. It is an unconditional assignment, i.e. it is connected continuously and not changed dynamically. It connects the value on the RHS to the output buffers described by the output mode variable on the LHS.

If we want to drive a three-state output pad one cycle after the assignment to **d** we need conditional output mode assignment. The code might be -

```
str(enable_pin, "D8");
var(din_pins, "[8]str", {"A1", "A2", "B1", "B2", "C1", "C2", "D1", "D2"});
```

```
var(dout_pins, "[8]str", {"E1", "E2", "F1", "F2", "G1", "G2", "H1", "H2"});
input(d_in, "int:8", loc=din_pins);
input(enable_in, "log", loc=enable_pin);
static(d, type(d_in), IOB=true);
static(enable, "log", IOB=true);
static(enable_del, "log");
static(dd, type(d));
output(out, type(dd),, loc=dout_pins);

d <- d_in;
enable <- enable_in;
enable_del <- enable;

when (enable)
    dd <- d;

when (enable_del)
    out |- dd;
```

where the last statement is a conditional target assignment, i.e. a three-state assignment. There is a library function **delayline()** which we could use instead of the assignment to an extra variable -

```
str(enable_pin, "D8");
var(din_pins, "[8]str", {"A1", "A2", "B1", "B2", "C1", "C2", "D1", "D2"});
var(dout_pins, "[8]str", {"E1", "E2", "F1", "F2", "G1", "G2", "H1", "H2"});
input(d_in, "int:8", loc=din_pins);
input(enable_in, "log", loc=enable_pin);
static(d, type(d_in), IOB=true);
static(enable, "log", IOB=true);
static(dd, type(d));
output(out, type(dd),, loc=dout_pins);

d <- d_in;
enable <- enable_in;

when (enable)
    dd <- d;

when (delayline(enable, 1))
    out |- dd;
```

5 Value mode variables

Consider the following code fragment -

```
static({a, b, c, d, e}, "int:8");
a <- c + d + e;
b <- c + d - e;
```

This creates five static variables, **a**, **b**, **c**, **d** and **e**, each of type "int:8", i.e. an 8-bit integer. The second statement assigns an expression containing **c**, **d** and **e** to **a** on every clock cycle. Similarly the third statement also assigns an expression to **b**. In each case expressions will be implemented by adders and subtracters but the 3PL compiler will not recognise the common sub-expression $c + d$. It is a waste of limited logic resources to duplicate the sub-expression so we need a way to extract the common sub-expression. A **value** mode variable provides the means to do this.

```
static({a, b, c, d, e}, "int:8");
value(v, "int:8");
v := c + d;
a <- v + e;
b <- v - e;
```

Here the value variable represents the expression $c + d$ which is thus only instantiated once, the output being shared by the two static assignments. The assignment operator for a value mode variable is `:=`. The assignment could be avoided by using the optional initialiser argument to the value() procedure -

```
static({a, b, c, d, e}, "int:8");
value(v, "int:8", c + d);
a <- v + e;
b <- v - e;
```

A value variable thus does not itself have state, i.e. there is no memory associated with a value variable. Note the following however -

```
static({a, b}, "int:8");
value(v, "int:8", b);
a <- v;
```

Here the value variable has as its value a single static variable. In this role it can be viewed as a pointer to that static variable (3PL does have pointer variables but these are not discussed here as they are seldom used in application code).

Value variables can be re-assigned at any time at which point they take on a new value. Value variables can also be evaluated prior to being assigned, in which case the later assignment is linked back by the compiler to the prior usage.

6 Parallel and sequential blocks

Parallel and sequential blocks provide the basic means of controlling execution order.

A sequential block is written as -

```
seq {
    statement
    ...
    ...
}
```

the enclosed statements being executed sequentially. The block terminates when the last statement in the block terminates. If the **seq** block has no enclosing conditional structure it will be executed repeatedly. If we wished to execute it once only, as in a conventional computer, we could terminate execution at the end of the block -

```
seq {
    statement
    ...
    ...
    stop();
}
```

A parallel block is coded in a similar way -

```
par {
    statement
    ...
    ...
}
```

the enclosed statements beginning execution simultaneously. The parallel block terminates when all enclosed statements have terminated, i.e. it terminates simultaneously with the longest executing statement(s). If the **par** block has no enclosing conditional structure it will be executed repeatedly.

These blocks may of course be nested.

Note that there is a difference between -

```
seq {
    a <- ...;
    b <- ...;
}
c <- ...;
```

and -

```
par {
    seq {
        a <- ...;
        b <- ...;
    }
    c <- ...;
}
```

In the first case the seq block executes repeatedly, taking 2 clock cycles each time. The single statement also executes repeatedly taking one clock cycle each time. In the second case the seq block and the single assignment start executing simultaneously but since the seq block takes two cycles the par block cannot terminate until the seq finishes. The par block then repeats, the seq block and the single assignment again starting simultaneously. Thus in the first case the two elements repeat independently whereas in the second case they repeat in step.

7 Target loops

Target loops, i.e. loops in the FPGA code, have the forms -

```
seq while (expression) {
    body
}
```

```
par while (expression) {
    body
}
```

```
seq do {
    body
} while (expression);
```

```
par do {
    body
} while (expression);
```

These function the same as conventional **while** and **do while** loops but with either sequential or parallel loop bodies. The braces are required even if the body is a single statement. Note that if the body is a single statement it does not matter whether the **seq** or **par** version is used.

These loops add no extra clock cycles on each iteration to the code body however there is one extra clock cycle required on loop termination.

8 Immediate execution

Thus far the language has been presented as a collection of statements to execute in the FPGA. It must appear a little odd that procedures are called to create variables, some of which (strings for example) do not appear to create anything in the FPGA. Where exactly do the procedures execute?

3PL is in fact a strictly sequential language - it is not a parallel language! 3PL source code is parsed into an internal form which is then executed by an interpreter. The language executes sequentially in the same way as languages such as C or Java. This execution is called **immediate** execution. Immediate assignment statements use the '=' operator. Immediate statements include **if()**, **for()**[1], **while()**, **do while()**, **switch**[2], **break**, **continue** and **return**. Assignments to value mode and priority mode variables and unconditional assignments to output mode variables are also immediate statements since they set up values within the interpreter and do not directly create in-line FPGA executable code. These all use the **:=** assignment operator.

During immediate execution 3PL encounters target statements which generate target code, i.e. logic destined for the FPGA. The target logic thus generated executes in the FPGA.

The following simple code -

```
int(i, 3);
for (int(j) ; j<3 ; j++)
    println(i + j);
```

is wholly immediate. The first statement creates an immediate integer variable **i** with an initial value of 3. The next statement is a **for** loop which creates an immediate integer variable **j** within the scope of the loop and iterates three times. On each iteration a line is printed to standard output consisting of the sum of the two variables, giving the output -

```
3
4
5
```

When this code is executed (immediate execution) no target logic at all is created. Except for the fact that variables are created by calls to procedures rather than in declarations, the immediate language is largely identical to C.

[1]The comma operator is not implemented. The initial section may contain a procedure call to create the loop variable which will be in the scope of the loop body

[2]Cases do not flow on as they do in C. Cases allow string constants.

The following code contains target statements (line numbers have been added as comments) -

```
int(i, 5);                          // 1
static({s1, s2}, "int:8");          // 2
par {                               // 3
    s1 <- i;                        // 4
    i += 7;                         // 5
    s2 <- i+2;                      // 6
}                                   // 7
```

Immediate execution of line 1 creates an immediate integer variable **i** with an initial value of 5. Execution of line 2 creates two static variables, **s1** and **s2**, which are 8-bit integers. This will result in target logic for two registers. Execution of line 3 sets up a pending parallel code block for the target. Execution of line 4 creates target logic for a static assignment, which is added to the pending parallel block. The RHS of the assignment is an immediate variable, so it is evaluated and thus the constant integer 5 is to be assigned in the target. When immediate mode values are encountered in target statements their current values at that point are used. Execution of line 5 adds 7 to the value of variable **i** but generates no target code. Execution of line 6 generates an assignment to static variable **s2** of the constant integer 14. This assignment is also added to the pending parallel block. At line 7 the pending parallel block is closed and the target statements encountered within it are linked in as parallel target execution threads. Since the parallel block is not enclosed in any other control structures it will be embedded in an infinite loop as usual.

It can be seen that intermediate execution provides fine control over what target code is generated. Further than that, intermediate execution is essential where an algorithm is required to generate the target algorithm. An extreme example is the well known Batcher parallel sort algorithm where a complicated algorithm must be executed to lay out the logic for the parallel sorting network. A rather simple nonsense example illustrates the idea -

```
var(m, "[]int", {3, 7, 1, 5, 9, 8});    // 1
static({s1, s2}, "[6]int:8");           // 2
par {                                   // 3
    for (int(i) ; i<6 ; i++)            // 4
        s1[i] <- m[i] * s2[i]           // 5
}                                       // 6
```

Execution of line 1 creates an immediate integer array initialised with arbitrary values. Execution of line 2 creates two static variables which are arrays of 6 8-bit integers. Execution of line 3 sets up a pending parallel target code block. Lines 4 and 5 constitute an immediate loop which on each iteration creates a target static assignment which is added to the pending parallel target block. Thus immediate execution has been used to iterate through a number of target code generation statements which are collected together for target execution in a parallel block. This code is equivalent to having written -

```
var(m, "[]int", {3, 7, 1, 5, 9, 8});
static({s1, s2}, "[6]int:8");
par {
    s1[0] <- m[0] * s2[0]
    s1[1] <- m[1] * s2[1]
    s1[2] <- m[2] * s2[2]
    s1[3] <- m[3] * s2[3]
    s1[4] <- m[4] * s2[4]
    s1[5] <- m[5] * s2[5]
}
```

In the examples shown so far the variable types have been fixed. Since the variable types are usually entered as a string (there is also a type "type", e.g. that is what the type() function returns) a type can be a string-valued expression, so we could paramaterise a type, e.g.

```
int(dim, 6);
int(width, 8);
static(s, "["+dim+"]int:"+width); // equivalent to "[6]int:8"
```

Here the type for static variable **s** is an expression constructed by concatenating string constants and variables (for strings '+' concatenates them). If the same type is to be used for several variables it could be assigned to a string variable, e.g.

```
int(dim, 6);
int(width, 8);
str(t1, "["+dim+"]int:"+width);
static(s1, t1);
static(sa, "[10]t1");
```

Note that a variable identifier can appear in a type string, as in the last line.

Immediate types do not have bit widths whereas target types must have a width. The exception is type "log" which does not have a width and can be used for immediate or target modes.

There is also a variable type "type". This can be created by several inbuilt procedures and functions. As mentioned the inbuilt function **type()** returns the type of a variable or expression (the function **typestring()** returns the type as a string). The inbuilt procedure **type()** (note that function and procedure call syntax is different so the same name can be used for these) creates a type from a string, so the above example could have been written -

```
int(dim, 6);
int(width, 8);
type(t1, "["+dim+"]int:"+width);
static(s1, t1);
static(sa, "[10]t1");
```

however **t1** is now not a string variable but a "type" variable.

Another procedure which creates a "type" variable is **struct()**. The call -

```
struct(s,
    f1, "log",
    f2, "[4]int"
);
```

creates a struct of immediate types, field **f1** being type "log" and field **f2** an array of four integers. This type can of course be nested, e.g.

```
struct(s,
    f1, "log",
    f2, "[4]int"
);
var(v, "[3]s");
v[0].f1 = false;
v[1].f2[3] = 5;
```

Structs can also be initialised, e.g.

```
struct(s,
    f1, "log",
    f2, "[4]int"
);
var(v, s, {true, {1, 3, 5, 7}});
```

The above example creates and initialises an immediate variable. The same can be done for a target mode variable, e.g.

```
struct(s,
    f1, "log",
    f2, "[4]uint:12"
);
static(v, s, {true, {1, 23, 570, 76}});
```

Where types are structs or arrays (or any nested combination) they may be assigned in one operation provided the LHS and RHS types match, including array dimensions. Thus the following is allowed -

```
struct(s,
    f1, "log",
    f2, "[4]int:3"
);
static(s1, "[6]s");
static(s2, "[3]s");
s2 <- s1[1..3];
```

where the notation [n..m] denotes a subrange, n and m being the limits. For a subrange the limits can be expressions. If the right hand limit is lower than the left hand limit the array members will be reversed.

Two other procedures which create types are **enum()** and **enumv()** which create enumerated types.

9 Target statements

Target statements create code which executes in the FPGA.

Target assignment statements use assignment operators which depend on the mode of the LHS variable. This is done so that the mode can be distinguished by sight rather than having to locate the variable creation procedure to determine its mode. These operators are -

- `<-` assign to a static mode variable
- `<<-` assign to a queue mode variable
- `|-` conditionally assign to an output mode variable

The target statement equivalent of an (immediate) **if ()** statement is **when ()**, which can also have an associated **else**. The true and false bodies can only be single target statements. These of course may be other target constructs such as **seq{}** or **par{}**, e.g.

```
when (enable)
    s <- 3;
else par {
    s <- 1;
    count++;
}
```

Different keywords for immediate constructs versus target constructs have been used to provide a visible distinction between immediate and target statements.

There is no target equivalent to the immediate **for ()** statement.

There are three target statement equivalents to an (immediate) **while ()** statement - **seq while () {}**, **par while () {}** and **sync while () {}**. The strange syntax is again intended to draw attention to the fact that these loops are target statements, not immediate. The **seq**, **par** and **sync** specify the loop body block type and the braces are required. As an example -

```
seq {
    count <- 10;
    par while (count != 0) {
        count--;
    }
}
```

will execute in 12 clock cycles. The initial assignment takes one cycle and a loop takes one cycle to exit in addition to the iterations (10 iterations in this case). Where the loop body has only one statement it does not matter whether **par** or **seq** is used.

There are also three **do while ()** loops - **seq do {} while ();**, **par do {} while ();** and **sync do {} while ();**, e.g.

```
seq {
    count <- 10;
    seq do {
        count--;
    } while (count != 0);
}
```

This loop will also take 12 clock cycles to execute.

10 Queue mode variables

A variable of mode **queue** is a queue whose default depth is two. A queue variable can be evaluated or assigned however any statement reading (evaluating) or writing (assigning) one or more queue variables will wait if a queue variable is not **available**. A queue variable is available for reading (evaluating) if it is not empty. A queue variable is available for writing (assigning) if it is not full. Otherwise queue variables are used in the same way as static variables. An expression can contain a mixture of static, value, queue and immediate mode variables. Any statement using that expression will wait if one or more queue reads within the expression are not available.

A queue write is done by a unique assignment operator -

```
queue({q1, q2}, "uint:8");
static(s, "uint:8");

q1 <<- q2 + s;
```

Queue variables may be any type.

Queue variables may be assigned at more than one place in the program however simultaneous assignments to the same element is erroneous and will give an unpredictable result[3]. Simultaneous assignments to different members of an array or different fields of a struct are usual. When a compound type (array or struct) of queue is assigned any fields or array members that are not explicitly assigned at the same time are assigned 0 for "int" or "uint" or false for "log".

Queue variables may be read at more than one place in the program and to describe the behaviour in that case we must first introduce the concept of a **module**.

A module is a block of code that has the following characteristics -

- it is a body of code that contains target execution threads each within its implied infinite loop
- it is a sink for queue reads

We looked earlier at the concept of statements being enclosed in infinite loops. It was stated that target statements that were not enclosed in any other structure were embedded in an infinite loop. A module is the environment in which all target execution execution threads originate. Each execution thread is initiated in an infinite loop, the loop iterating at the completion of its thread. Every 3PL program has a main module which is associated with the main source file, i.e. the source file passed to 3PL when it was called. This module [4] contains the infinite loops enclosing enclosing statements in that source file.

It is crucial to establish sinks for queue reads, and a module is a queue sink. A queue which is read in more than one queue module will not pop the datum until all modules have executed a read. Thus if a queue is read in two modules if one module reads the queue but the other does not, the queue becomes unavailable in the first module and remains so until the second module has read the queue. When all modules have read a queue the datum is removed and if another datum is present in the queue queue the queue becomes available again to all modules. Thus a queue can have multiple sinks but each must read a datum before the queue pops that datum and makes the next (if any) available. This data synchronisation occurs automatically. Note that if all queue sinks (modules) read simultaneously as soon as read availability occurs then the queue can transfer data at the clock rate - there is no time penalty in this mechanism.

[3] There is a priority variable mode which can be used in arbitration code.
[4] it is called a **file** module which is described later

Within a queue sink (module) there can be multiple reads of the same queue but the effect these have on the queue depends on timing. If one or more queue reads of a queue occur simultaneously within a module that is regarded as a single read and each one receives the same datum. If multiple reads of queue within the same module occur on different clock cycles each of these is a separate read and they will read different data. Each read may then also wait for other queue sinks (modules) to complete their reads before advancing to the next datum.

There are two types of module -

- a named module which is called in the same way as a procedure, except that the call does not produce in-line code, and the definition has the keyword "module" instead of "procedure"
- an un-named module which is simply code enclosed in square brackets

A named module call or an un-named module instance do not produce code in-line at their location but instead establish a separate environment for execution threads. A module can contain any number of target statements and each statement will be enclosed in an infinite loop.

Named modules have their own local scope in the same way as procedures. Un-named modules also have their own scope, but they can also access the scope in which they are embedded. This is because they are not passed arguments so they need access to the local scope to access variables.

The following example shows the use of a named module with queue reads -

```
queue({q1, q2}, "uint:4");

q1 <<- .....

q2 <<- q1;
...... q2;

latchout(q1);

module
latchout (queue(p)) {
    var(opins, "[]str", {"A1", "A2", "B1", "B2"});
    static(s, type(p));
    output(,, s, loc=opins);
    s <- p;
}
```

The main line program code is in the module associated with the source file (called a **file** module). This code writes some value to queue **q1**. Queue **q1** has two sinks - the current module, where it is read and assigned to **q2**, and the module **latchout** where it is read and assigned to a static variable which is connected to output pins. Thus the queue **q1** will expect reads from two modules, the file module and module **latchout** and will deliver the same datum to each regardless of when each sink decides to execute a read.

We could rewrite the above example using an un-named module -

```
queue({q1, q2}, "uint:4");

q1 <<- .....

q2 <<- q1;
...... q2;

[
    var(opins, "[]str", {"A1", "A2", "B1", "B2"});
    static(s, type(q1));
    output(,, s, loc=opins);
    s <- q1;
]
```

Here we use variable **q1** directly where previously it was passed as an argument to the named module. We can do so because an un-named module can access variables in the scope in which it is enclosed

(compare this with a seq block or a for loop where again statements within those constructs can access variables in the scope in which they are enclosed).

It can be seen that the timing of queue accesses is very important. The time at which queue reads or writes are attempted (depending on availability) may not be deterministic, i.e. it may depend on external input. To synchronise queue operations there is another target control structure **sync{}**. This is the same as **par{}** except that the parallel block will not begin execution until all queue reads and queue writes within the block can proceed. Thus a sync block containing multiple statements involving queue reads and queue writes guarantees that on entry these statements can execute in one clock cycle. A sync block can also be used to ensure that other target statements execute at the same time as queue operations, for example to count queue accesses -

```
queue(p, "uint:4");
static(count, "uint:32");

par {
    .... <- p;
    count++;
}
```

is not correct since the count may advance by one before the queue is available. If this is rewritten -

```
queue(p, "uint:8");
static(count, "uint:32");

sync {
    .... <- p;
    count++;
}
```

the count increment will occur in the same clock cycle as the queue read.

For target loop statements there is a **sync while** and **sync do while** where the loop bodies are sync blocks instead of par blocks or seq blocks.

Queue variables can be tested for availability. The unary operator `<` is true if its queue operand is available for reading. Similarly the unary operator `>` is true if its queue operand is available for writing. A queue can be examined instead of read by using the unary operator `?`. This will return the current datum from the queue without popping the datum or interacting with any other queue sinks in any way. This can be used for example to select a datum from one of two queues without reading both, e.g.

```
queue({q1, q2, p3}, "uint:8");

sync {
    q1 <<- (?q2 > ?p3) ? q2 : p3;
}
```

which examines queues **q2** and **p3** and reads whichever has the largest datum, assigning it to queue **q1**. The sync ensures that **q2** and **p3** are available for reading and **q1** is available for writing before attempting to execute the statement (the `?` unary queue examine operator does not wait for read availability and so could get a spurious value).

It can be seen that there are two, orthogonal, target execution control mechanisms in action. Statement execution is primarily initiated via threads within modules. For each statement, execution may be delayed until queues are available. Systolic code can be written where every statement has its own execution thread and execution depends solely on queue availability. This form of coding could perhaps be described as data flow. For example the following illustrates how this might appear -

```
type(t, "int:16");
var(ipins, "[3][8]str",
    { {"..", "........"..."},
      {"..", "........"..."},
      {"..", "........"..."}  }
```

```
);
var(iepins, "[3]str", {"..", .... , ".."});
var(opins, "[8]str", {"..", ........"..."});
str(oepin, "..");

queue({q1, q2, p3, p4, p5, p6, p7, p8}, "uint:8");

datain(q1 <- ipins[0], iepins[0]);
datain(q2 <- ipins[1], iepins[1]);
datain(p3 <- ipins[2], iepins[2]);

[p4 <<- q1 + q2;]
[p5 <<- q1 - q2;]
[p6 <<- p4 - p3;]
[p7 <<- p5 + p3;]
[p8 <<- p6 + p7;]

dataout(p8, opins, oepin);

module
datain (queue(p) <- var(pins, "[]str"), str(epin)) {
    input(data, type(p), loc=pins);
    input(enable, "log", loc=epin);
    static(d, type(p));
    static(e, "log");
    d <- data;
    e <- enable;
    when (e)
        p <<- d;
}

module
dataout (queue(p), var(pins, "[]str"), str(epin)) {
    static(data, type(p));
    static(enable, "log");
    data <- p;
    enable <- <p;
    output(,, data, loc=pins);
    output(,, enable, loc=epin);
}
```

This example takes three 16 bit inputs and performs a series of pipeline operations upon them until they emerge combined into a single output. The inputs and outputs are latched using static variables - this is the safest way of clocking data on and off chip [5]. It may not be strictly necessary but for safety each of the queue assignment statements has been placed in an un-named module to ensure that multiple queue reads are handled as separate queue sinks. In fact since queue reads would occur simultaneously on queue availability in each case this is not necessary, but is good practice and it saves having to think it through. Notice that no execution control structures were needed for this processing pipeline. Because the pipeline has data paths which diverge and then recombine there is the potential for parallel alternative data paths to deleteriously interact. Alternative pipeline data paths can interact if they do not have identical latency (in the example above they do). The following example has this problem -

```
queue({q1, q2, p3, p4}, "uint:8");

mod1(q2 <- q1); // assume this module has 3 pipeline stages
mod2(p3 <- q1); // assume this module has 6 pipeline stages
p4 <<- q2 + p3;
```

Here the first shorter path will fill queue **q2** after 5 clock cycles (assuming **q1** is always available). The last statement cannot execute because **p3** is not yet available, the second pipeline path being longer.

[5] The registers will be located in the I/O pads by default if the correct Xilinx map option is used. Alternatively the iob attribute for the static variables could be set true.

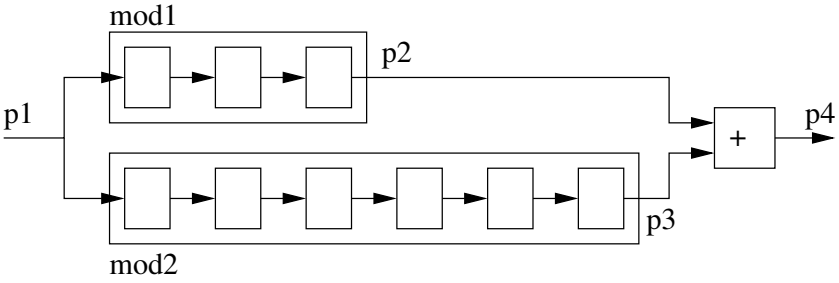

Figure 1: alternate pipeline paths

The buffer in q2 is now full so it becomes unavailable for writing and the same will apply through the first pipeline. But this means that the first pipeline will stop reading **q1** which means that the second branch has not yet reached the output but cannot continue to read **q1**. The data already in the second pipeline will continue to move through the queue but no more input to that queue will occur. When the data in the second pipeline reaches the output the last assignment statement can now execute until it exhausts the second pipeline. The two pipeline branches are interacting and this will slow throughput to about half. This interaction can be eliminated by making the queue buffer depth of queue **q2** large enough to keep accepting data until the second pipeline fills. The example would be fixed by -

```
queue({q1, q2, p3, p4}, "uint:8");
attributes(q2, depth=16);

mod1(q2 <- q1); // assume this module has 3 pipeline stages
mod2(p3 <- q1); // assume this module has 6 pipeline stages
p4 <<- q2 + p3;
```

The buffer size is rounded up to various powers of two. Which powers of two, and the maximum allowed, depend on the FPGA - see the 3PL manual.

11 Target exceptions

3PL is designed for the generation of code which will process continuous data streams. If an exception condition arises it may prove necessary to prematurely exit from a conditional or loop block. In particular a statement within a block may be waiting for queue availability where the exception condition is such that this will not occur. A way is needed to force an exit from loops, code blocks or blocked statements.

The following target constructs may have an **unless** clause attached -

- **seq {}**
- **par {}**
- **sync {}**
- **when ()**
- **when () else**
- **seq while () {}**
- **par while () {}**
- **sync while () {}**
- **seq do {} while ();**
- **par do {} while ();**
- **sync do {} while ();**

For example a seq block may have some queue reads which we want to terminate when an exception condition occurs -

```
static({s1, s2}, ...);
queue({q1, q2}, ...);
value(e, "log", ...);

seq {
    s1 <- q1;
```

```
        s2 <- q2;
    } unless (e)
```

The **seq** is at the top level and will execute in an infinite loop. If **e** is true the **seq** will immediately terminate on the next clock edge. If any assignments in the **seq** were ready to execute on that clock edge they will do so. Following termination the normal sequence of execution of the thread will continue, in this case the **seq** immediately executing again. This is probably not what we want - there is usually some corrective code we wish to execute. To do this there is an optional execution block associated with the **unless**, e.g. -

```
static({s1, s2}, ...);
queue({q1, q2}, ...);
value(e, "log", ...);

seq {
    s1 <- q1;
    s2 <- q2;
} unless (e) {
    reset(q1, q2);
}
```

The exception code body is in braces and may contain a number of statements, however it can only contain one target statement. That target statement may of course be a **seq {}**, **when ()**, **par while () {}** etc. The exception code block is executed after the terminated block, following which normal thread execution will continue.

Target statements with **unless** clauses cannot be nested[6].

12 combinatorial output memory

Combinatorial output memory can be used as a combinatorial function on read, i.e. when an address is present at the input the memory output will reflect the value stored at that address after a short delay. A memory read does not require a clock. A memory write is clocked. In the case of Xilinx FPGAs combinatorial output memory is distributed CLB RAM.

Combinatorial output memory is accessed by means of variables whose mode is **cmemory**. These can only be created by the inbuilt procedure **cmemory()**, e.g.

```
int(ports, 2);
str(dtype, "uint:8");
str(atype, "uint:6");
var(init, "[]uint", {.., .., .., .., ..    , ..});
cmemory(m1, ports, atype, dtype, init);
```

The first argument to **cmemory()** is the identifier for the created variable. The second argument is the number of ports which for Xilinx can be 1 or 2. The address type and data type follow. These can be any type at all but the address type is restricted to a maximum width of 9 bits. The last argument is initialisation data for the memory and is optional. The initialisation argument is shown in the example as an array variable but it could be a list of values in braces. Where an array variable is used it could of course contain values computed in an immediate execution loop, e.g. values implementing a mathematical function or a translation table.

If the memory has more than one port the ports can be accessed independently. In the case of Xilinx the second port, port 1, is read-only. The ports can be accessed at multiple places in the code but the code must be written to ensure that access on a port is mutually exclusive as access conflicts are not detected or resolved.

Because the memory behaves as a combinatorial element on reading, the memory output is accessed via an inbuilt function, e.g.

[6]Nesting of target statements with **unless** clauses may be allowed in the future if and when I can figure out the complicated logic to do it.

```
static(d, dtype);
static(a, atype);
static(l, "log");

when (l)
    d <- cmemoryread(m1, 0, a) + 3;
```

The memory data is available in the same clock cycle that the read occurs because it does not have to wait for a clock - the address propagates through the memory emerging as the stored data element at the output.

This example uses static variables. When **l** is true **d** is assigned the sum of the memory output from port 0 at address **a** and the constant 3. We could also use queue variables, e.g.

```
float(gamma, 0.45);
float(gain, pow(256.0, 1.0 - 1.0 / gamma));
str(dtype, "uint:8");
var(init, "[256]uint");
for (int(i) ; i<256 ; i++)
    init[i] = round(gain * pow((float)i, 1.0 / gamma));
cmemory(gamma_correct, 1, dtype, dtype, init);
queue({q1, q2}, dtype);

.
.
q2 <<- cmemoryread(gamma_correct, 0, q1);
.
.
```

where the memory read occurs whenever queues **q1** and **q2** are available (writing of **q1** and reading of **q2** are not shown in the code fragment). This example purports to show an 8-bit pixel gamma correction (I may have the function wrong, but you get the idea). The gamma correction lookup table is a cmemory mode variable with one port. The address input and data output are each 8 bits wide. The initialisation array has its contents computed during immediate execution using the value initially assigned to floating point variable **gamma**. The floating point values are rounded back to the nearest integer for assignment to the initialisation array[7]. A variation is to use a two-port cmemory and allow table entries to be overwritten from an external CPU when required. The Zynq platform provides this facility as do many simple FPGA platforms with a serial interface from a USB port. In these cases the above code might then be written as -

```
class(cpu, cpu_api());
str(dtype, "uint:8");
cmemory(gamma_correct, 2, dtype, dtype); // 2 ports, separate for read and write
queue({wdata, waddr, q1, q2}, dtype);
cpu.write(wdata <- "WD");
cpu.write(waddr<- "WA");

cmemorywrite(gamma_correct, 0, waddr, wdata);

.
.
q2 <<- cmemoryread(gamma_correct, 1, q1);
.
.
```

The first line creates a CPU interface class. The class procedure **write()** allows a CPU coupled to the FPGA to write to a queue variable in the FPGA. The input arguments **"WD"** and **"WA"** are string identifiers for the bus interface to the FPGA. In this example the **cmemorywrite()** inbuilt procedure will write to the memory when queues **waddr** and **wdata** are both available.

[7] 3PL provides target floating point format but arithmetic operations are logic-intensive and must be pipelined, i.e. FP operations cannot be done in one step in one clock cycle. There is a library of pipelined floating point operations. Target fixed point types could also be used, rather than integer

Of course we could combine a more complicated execution thread with queue variables -

```
struct(s,
    sum,    "uint:16",
    count,  "uint:16"
);
queue(dq, s);
queue(aq, atype);
static(count, "uint:16");
static(accum, "uint:16");

seq {
    sync while (aq != 0) {
        accum <- accum + cmemoryread(m1, 0, aq);
        count++;
    }
    sync {
        dq.sum <<- accum;
        dq.count <<- count;
        count <- 0;
        accum <- 0;
    }
}
```

Here we loop reading memory and accumulating the sums of the values until the queued address is zero, at which point the sum and count are written to a queue (whose type is a struct containing fields for both values) and the static variables are cleared for the next iteration. The seq block then repeats.

In this last example if the input queue **aq** transmits all zeros the output data stream **dq** will be at the same rate, otherwise the output will be at a lower rate which depends on the length of non-zero sequences. This is a characteristic of a systolic network with buffered queues, compared with a normal signal processing lock-step approach, that each datum can proceed through the network at a rate determined by producers and consumers at each node of the network. Where all inputs to the network have data available and each node can execute in one clock cycle then data can flow at clock rate. Where data input is sporadic or at sub-clock-rate, or multiple sequential execution cycles are needed at some nodes, the data throughput will be slower.

13 registered output memory

Registered output memory uses a clock for both read and write. On executing a read the data appears at the output following the next clock edge. In the case of Xilinx FPGAs registered output memory is block RAM.

Because the memory output is delayed one clock cycle on a read, registered output memory reads must be pipelined, otherwise a read will take two clock cycles.

Registered output memory is accessed by means of variables whose mode is **rmemory**. These can only be created by the inbuilt procedure **rmemory()**.

In the previous section we had an example where data bytes were streamed through a look-up table for gamma correction -

```
type(dtype, "uint:8");
cmemory(gamma_correct, 2, dtype, dtype);
queue({wdata, waddr, q1, q2}, dtype);

.
.
q2 <<- cmemoryread(gamma_correct, 1, q1);
.
.
```

We could convert that example to use registered output memory as follows -

```
type(dtype, "uint:8");
rmemory(gamma_correct, 2, dtype, dtype);
queue({wdata, waddr, q1, q2}, dtype);

.
.
seq {
    rmemoryread(gamma_correct, 1, q1);
    q2 <<- rmemoryout(gamma_correct, 1);
}
.
.
```

The seq loop will first attempt to execute a memory read from port 1 taking the address from queue **q1**. If **q1** is not available then the read will wait until it is. When the read has occurred the seq block will then execute the assignment of the current output value of the memory to queue **q2** (if **q2** is not available it will wait). The inbuilt function **rmemoryout()** gets the contents of the output register of the block RAM for the specified port. This value will remain in the output register until another memory read is executed. This code example does not pipeline the memory reads and hence takes two cycles per access.

Note that in this example **rmemoryread()** is a procedure, not a module, so the module in which **q1** is being read is the file module. Since there is only one sink for queue **q1** we do not need to worry about ensuring that bits of the code are in separate modules and hence all the code is in the file module. If queue **q1** was also read by other unrelated code we would have to ensure that the reads were in separate modules so that the independent reads were handled correctly.

To pipeline the above example the memory read must be separated from the following memory output value assignment. The assignment is dependent on a previous memory read so we need to signal that this has occurred. This could be done with another queue variable -

```
type(dtype, "uint:8");
rmemory(gamma_correct, 2, dtype, dtype);
queue({wdata, waddr, q1, q2}, dtype);
queue(read, "log");

.
.
sync {
    rmemoryread(gamma_correct, 1, q1);
    read <<- true;
}

when (read)
    q2 <<- rmemoryout(gamma_correct, 1);
.
.
```

but this code is not correct! If queue **q2** becomes unavailable for writing the assignment to it will wait but the next memory read may go ahead, overwriting the previous registered memory output value. We could acknowledge back via another queue but that would introduce an extra clock cycle.

We need to execute the memory read and assign the result in the one clock. Since the read must occur in a clock cycle previous to the assignment to q2 the two operations can be staggered by doing an initial read prior to entering the loop.

```
type(dtype, "uint:8");
rmemory(gamma_correct, 2, dtype, dtype);
queue({wdata, waddr, q1, q2}, dtype);

.
```

```
.
seq {
    rmemoryread(gamma_correct, 1, q1);
    sync {
        q2 <<- rmemoryout(gamma_correct, 1);
        rmemoryread(gamma_correct, 1, q1);
    }
}
.
.
```

here -

- the first statement in the **seq** block executes the first read
- the second statement in the **seq** is an infinite loop
- the loop sync block will not execute until **q1** and **q2** are available (**q1** is not empty and **q2** is not full)
- **q2** is assigned the previous result and the next read are performed simultaneously
- the **seq** block never completes

ANother approach is to create a module to handle pipelined rmemory reads -

```
type(dtype, "uint:8");
rmemory(gamma_correct, 2, dtype, dtype);
queue({wdata, waddr, q1, q2}, dtype);
queue(read, "log");

.
.
rread(q2 <- gamma_correct, 1, q1);
.
.
```

The code for the module might be -

```
global module
rread (queue(data) <- rmemory(m), int(port), queue(addr)) {
    static(hold, "log");
    when (!hold || >data)
        rmemoryread(m, port, addr);
    when (hold)
        data <<- cast(rmemoryout(m, port), type(data));
    hold <- <addr || hold && !>data;
}
```

We have declared the module as **global** so that it can be used anywhere - the scope of variables, modules, procedures and functions is discussed in a later section. In this module a static boolean variable **hold** is used to synchronise the memory reads and output queue assignments. When **hold** is true a value is held in the memory output register waiting to be assigned to the output queue. There are three statements in this module, each one executing independently in its own implicit infinite loop.

The first statement executes a memory read if the hold flag is false (there is currently no unassigned registered memory output value) or if the output queue is available for writing (the > unary prefix operator returns true if the queue variable to which it is applied is available for writing - there is a similar operator, < for read availability). If <q1 is not true the statement will wait until it is true.

The second statement assigns the registered memory output value to the output queue when **hold** is true. If the output queue **data** is not available this statement will wait untli it is available then execute the write.

The third statement continually updates the value of **hold** - it is assigned true if an input is available or it is already true and the output queue is not available, otherwise it is assigned false.

This module is already included in a 3PL library.

14 Clocks

A clock is another mode of variable - mode **clock**. A Clock variable can represent -

- a clock input from an input buffer driven from an FPGA pad
- a clock signal between a digital clock manager and a global clock buffer
- a clock signal distributed over a clock net driving FPGA synchronous logic

A clock mode variable is created by calling the inbuilt procedure **clock()**. For an input clock signal various attributes such as the frequency and the FPGA pin(s) must be supplied. For example -

```
str(c100_pin, "E17");
clock(c100_in, frequency=1.0e8, loc=c100_pin);
```

If the clock variable does not represent the output of an input clock buffer then there is no location attribute.

If a clock is passed as an argument to a module, procedure or function then the parameter is not given any attributes, e.g.

```
procedure
p (.... <- clock(c)) {
    .
    .
}
```

To generate a clock, a digital clock manager (DCM) is used. The following code illustrates this -

```
clock(c100_in, frequency=100.0, loc="E17", ibufg=true);
clock({global.c100, c100_raw});
clkdcm(clk0=c100_raw <- clkin=c100_in, clkfb=c100);
clockbuffer(c100 <- c100_raw);
```

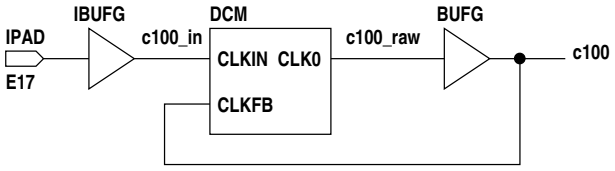

Figure 2: digital clock manager

Creating c100 with global scope ensures that it is globally accessible where **c100_in** and **c100_raw** may have local scope. Library procedures **clkdcm** and **clockbuffer** both propagate the frequency and period attributes from inputs to outputs so we don't have to assign these attributes to **c100_in** and **c100_raw**.

Before any logic involving synchronous elements is generated by 3PL the clock domain must be established. This is done by calling the inbuilt procedure **useclock()**, e.g.

```
useclock(c100);
```

When the clock domain is changed using **useclock()** the previous clock domain is pushed onto a stack. If **useclock()** is called with no argument the previous clock domain is popped off the stack and again becomes the current clock domain.

The clock domain used by variables is established when they are assigned or evaluated, not when they are created.

For a static variable the current clock domain when the first assignment statement in which the variable appears (either LHS or RHS) is encountered establishes the clock domain for the variable. Any other assignments to or evaluations of that variable must be in the same clock domain.

For a queue variable the read and write clock domains may be different. The write clock domain is established as the current clock domain when the first write to the queue is encountered and all

subsequent writes must be in the same clock domain. Similarly the read clock domain is established as the current clock domain when the first read of the queue is encountered and all subsequent reads must be in the same clock domain. If the read and write clock domains are the same, which is the usual case, the queue is called **synchronous** and the queue buffer depth will be 2 by default. If the read and write clock domains differ the queue is called **asynchronous** and the default buffer depth is 16. Resynchronisation of queue data between the two clock domains is handled automatically by the asynchronous queue.

In general in a statement the clock domain in use must agree with the clock domain of variables that are assigned or evaluated in that statement. Procedure and module calls are an exception in that the clock domains are checked in the body of the called module or procedure but not when arguments are evaluated and passed to parameters.

The correct way to pass data from one clock domain to another is via an asynchronous queue, however there are two functions, **accept()** and **sample()** which allow mixing of clock domains at the programmer's peril. **accept()** simply overrides the clock checks and can be used if data is seldom changed and the risk of data corruption is acceptable (safe for single bit data except for the very small probability of getting a metastable state. For single bits an inbuilt function **resync()** can be used to do this properly). **sample()** preserves data integrity by resampling it into the current clock domain, however since the clock frequencies differ a datum may be lost or duplicated (but will not be corrupted).

15 Modules

Modules and procedures were briefly mentioned in the context of queue variables. A module has the important property that it delineates a sink for queue variables. When a Queue has more than one sink each sink must execute a read on the queue before the datum is popped and the next becomes available. Multiple reads within a sink are treated as a single read if they occur simultaneously, otherwise they are separate reads. A queue has two sinks if the two code sections which read the queue are in different modules, For example -

```
queue({p, q1, q2}, t1);

.
q <<- .....;
m1(q1 <- q);
m2(q2 <- q);
.

module
m1 (queue(out) <- queue(in)) {
    out <<- in - 3;
}

module
m1 (queue(out) <- queue(in)) {
    out <<- in * 2;
}
```

Here when either of the modules reads its input queue, popping a datum, the queue will appear to be no longer available (empty) until the other module also executes a read. If both reads occur simultaneously then the next queue datum will immediately be available to both modules. Thus queue reads in multiple sinks will all get the same datum, subsequent reads waiting until all modules have been satisfied.

Where there are multiple read statements within one module -

- The reads are independent of each other.
- The first queue read to occur will extract a datum as described above. Any subsequent queue read by any these statements will wait until all other queue sinks (modules) have read that datum.
- If one or more queue reads can execute simultaneously, i.e. their input and output queues are all available, then they will execute in the same clock cycle and will read the same datum
- If queue reads occur at different times then each will pop different (successive) data.

It is common to use separate statements to read different fields where the data type is a **struct** and in this case it is usual to enclose these in a **sync{}** to ensure they execute in the same clock cycle.

The above is illustrated in examples following.

Modules are declared with output parameters followed by input parameters, these lists being separated by **<-** (the left arrow can be omitted if there are no output parameters). Each parameter is a call to an inbuilt variable creation procedure. References to parameters in the module code body are replaced by the arguments passed in the call. If a parameter is not passed an argument then it becomes a local variable. For this reason parameters can have initial values which become the variable initial values if no matching argument is provided. There is an inbuilt procedure **matched()** which can be used to test if a parameter has been passed an argument. Arguments are normally matched to parameters by the order in which both occur however an alternative is to use **parameter=argument** pairs in the call. This is useful where there are many parameters and only a few are to be passed arguments. A combination of both modes is allowed as long as no positional arguments follow any parameter=argument pairs.

Parameters can have target type widths missing, dimensions missing or even a type missing or a mode missing. The missing elements will be filled in from the argument. Where parameters have types and dimensions these will be checked against the argument type and dimensions. An exception is target numeric type widths which allow a mis-match, resulting in truncation or padding.

A module is a block of code which can be called. Module calls are only done during immediate execution. There is no code in the FPGA which executes any sort of shared code call! When a module is called however any target statements that it contains are established as independent execution streams. A module call would not usually be made within a target construct such as a **when** or one of the target loops. In contrast a procedure containing a target statement called within a target construct will result in that statement appearing in-line where the call is made.

Declaring a module and calling it can be verbose and inconvenient, especially if the code blocks are small and the modules are only called once. Instead an un-named module can be used. An un-named module is code enclosed in square brackets. It is not called as such but simply appears in-line, though it is executed as a distinct and separate module from that within which it is nested. The above code could be written

```
queue({q, q1, q2}, t1);

.
q <<- .....;
[q1 <<- q - 3;]
[q2 <<- q * 2;]
.
```

where now the assignments to queues **q1** and **q2** are in separate modules - the code in brackets. Un-named modules do not have calls and hence there are no arguments and parameters. The scope of an un-named module is like that of a code block in braces. Variables created inside the brackets are in the scope of the un-named module. But code within the un-named module can access the scope in which it is enclosed - in the case above **q**, **q1** and **q2** occur in the scope outside the two un-named modules but are accessible within the un-named modules.

Note that in this example the assignment to **q1** will occur when both **q1** and **p** are available. Similarly the assignment to **q2** will occur when both **q2** and **q** are available. If **q1** and **q2** become available at different times the assignments will not occur simultaneously, but each will read the same datum from **p** since **p** will not pop its datum until both reads have occurred (the first read to occur will not see **q** become available again until after the second read has occurred). If instead the code was

```
queue({q, q1, q2}, t1);

.
q <<- .....;
q1 <<- q - 3;
q2 <<- q * 2;
.
```

it would function correctly if all three queues were available, both **q1** and **q2** being assigned the datum from **q**. If however **q** and **q1** were available but **q2** was not, the assignment to **q1** would take place and the datum would be popped from **q**. When **q2** became available it would read the next datum from **q**. Another solution might be

```
queue({q, q1, q2}, t1);

.
q <<- .....;
sync {
    q1 <<- q - 3);
    q2 <<- q * 2);
}
.
```

which inhibits both assignments until all three queues are available.

16 Modules, procedures, functions and classes

3PL provides user-defined modules, procedures, functions and classes.

Briefly -

- module, procedure, function and class calls are only executed during immediate execution - the calls are not executed in the FPGA
- modules and procedures are called using the same syntax and semantics
- modules and procedures are declared the same way except for the keyword **module** or **procedure**
- a module may contain any number of target statements whereas a procedure can contain at most a single target statement (but which may be a seq, par sync etc.)
- each target statement in a module is established in its own execution thread whereas the single target statement in a procedure is executed in-line at the location of the call
- modules and procedures with no target statements are functionally identical
- modules delineate queue sinks but procedures do not
- functions can only appear within the context of an expression - they cannot be called using module or procedure call syntax
- functions can return immediate or target values
- functions cannot contain in-line target statements - they can contain module calls
- classes are called in a similar manner to modules or procedures but they return a value, a class instance
- a class cannot contain in-line target statements

A procedure call and definition are illustrated by the following example -

```
static(s, "uint:8");    // unsigned integer of 8 bits
static(v, "[2]uint:8"); // array of 2 unsigned integers of 8 bits

p(s <- v[0], v[1] + 2, 3);

procedure
p (static(os, "int:8") <- value({iv1, iv2}, "int:8"), int(i)) {
    os <- iv1 + (iv2 >> i);
}
```

This procedure, **p**, has one output parameter, the 8-bit integer static mode variable **os**, and three input parameters, two value variables and an immediate integer variable. Parameters are created by using the same variable creation procedures as before.

Procedures have their own variable scope so any variables created in the body of the procedure (and any unmatched parameters) are in the local scope of the procedure and cannot be accessed from outside the procedure, including any deeper calls to procedures (or functions).

The convention for both procedure definitions and calls is that parameter and argument lists consist of outputs, `<-` and then inputs. If there are no output parameters in the definition the `<-` can be omitted.

Similarly if there are no output arguments in the call the <- can be omitted. For value and immediate mode input parameters the argument is copied to the parameter by assignment, i.e. it is call-by-value. For other input parameter modes and for all output parameters the parameter 'points' indirectly to the argument variable, i.e. call-by reference.

A value mode input parameter can be passed an argument which is an expression, and that expression could be immediate mode (i.e. a constant in the target). Similarly an immediate mode input parameter can be passed an argument which is an expression, however that expression must be immediate mode, not a target mode. In the example above the first input parameter is value mode and has been passed one member of a static mode array. The second value mode parameter has been passed an expression which is the sum of a member of a static mode array and a constant. The body of the procedure adds the first parameter to the second parameter right shifted by the third and assigns the result to the output static variable.

For the other cases these are call-by-reference and any reference to the parameter accesses the actual variable passed as an argument. Note that where the type is an array the argument may be a sub-array, e.g. -

```
static(v, "[6]uint:8");

p(v[1..3]);

procedure
p (static(p, "[3]int:8")) {
    ...
}
```

This applies to call-by-value and call-by -reference and to both input and output parameters.

If parameter dimensions are given the argument dimensions must be the same, however we could omit the parameter dimensions to make the procedure more flexible -

```
static(v, "[6]uint:8");

p(v[1..3]);

procedure
p (static(p, "[]int:8")) {
    println(dimensions(p));     // print the number of dimensions of p
    println(dimension(p, 0));   // print the first dimension of p
}
```

which would print

```
1
3
```

Arguments may be omitted in which case the unmatched parameters become local variables in the procedure. Input parameters may be given initial values so that these values will be used if no matching argument is passed.

Arguments are assigned to parameters in left to right order with inputs assigned before outputs. This means that when a parameter is created the type argument to the inbuilt variable creation procedure could use values passed to parameters to the left.

A parameter can be tested to determine if an argument has been passed to it by means of the inbuilt function **matched()** which returns true if an argument has been passed.

In addition to the usual matching of arguments to parameters by position the keymatch format may be used where each argument is identifier=value, the identifier being that of the parameter. The keymatch arguments can be in any order but arguments are assigned to parameters in parameter order. Positional arguments can precede keymatch arguments but cannot follow them.

Considerable flexibility is allowed in argument passing. Missing parameter dimensions can be inferred from the argument dimensions and target type widths need not be the same (truncation or padding will result if not).

Procedures may contain immediate statements however they may only contain at most a single in-line[8] target statement! This statement may be a control structure containing other target statements nested to any depth. The following procedure definition -

```
procedure
p () {
    a <- 3; //
    b <- 4; // ILLEGAL!
}
```

is not allowed. It is not clear what is to be done with two target statements. If **p()** is called at the top level you might expect the assignments to be placed in their own infinite loops. If called in a par or seq block then perhaps they are added to the block. To avoid this uncertainty procedures can only contain one target statement but of course this can be a **par{}**, **seq{}**, **when**, **seq while**, **par while** etc.

Functions differ from procedures in that -

- there are no output parameters
- there are no in-line target statements
- a function call appears where a value is required, not on a line by itself as for a procedure call.
- there is a return statement to return a value - this can only be immediate mode or value mode

Functions have local variable scope in the same way as procedures.

Functions cannot have target statements but they can have target expressions and return values. The following illustrates a trivial function returning a target mode value -

```
function
add (value({a, b}, "int:8")) {
    value(v, "int:9")
    v := a + b;
    return(v);
}
```

Note that the declaration does not define the return type, so a function could have several different return types governed by conditional statements.

The trivial example above is verbose and could be recoded as -

```
function
add (value({a, b}, "int:8")) {
    value(v, "int:9", a + b)
    return(v);
}
```

or simply -

```
function
add (value({a, b}, "int:8")) {
    return(a + b);
}
```

It was previously stated that procedures could contain at most one in-line target statement and functions could not contain any. The term "in-line" excludes statements contained in modules within a procedure or functions. The reason is that a module is an independent block of code that originates target execution threads. If a procedure contains a target statement that statement is in-line, i.e. that statement will be inserted into the target execution thread in the context where the procedure is called. When a module is called it establishes a completely new context for establishing execution threads unrelated to the context in which it is called. As an example suppose we wish to write a function which returns a value which

[8] This qualification will be explained later.

is the value of the argument delayed by a number of clock cycles. The function cannot contain target statements so it seems that we could not write this, but a function can contain a module which in turn can contain target statements. We could write the function as follows -

```
global function
del (value(in), int(n, 1)) {
    type(t, type(in));
    static(d, "["+n+"]t");

    [
        d[0] <- in;
        for (int(i, 1) ; i<n ; i++) // loop for i from 1 to n-1
            d[i] <- d[i-1];
    ]

    return(d[n-1]);
}
```

The first input parameter has no type specified so it will use the type of its argument. The second argument is immediate and is the delay count. It is set to default to 1 if it is not given an argument. Note that the array type is constructed as a string expression to insert the dimension gained from the second parameter. The value returned for the function is value mode and is the value of the last member of the static variable array. Note that the loop starts at index 1 as the loop variable **i** is initialised to 1.

The un-named module can access the scope of the function in which it is contained. Note that the assignments to variable **d** are not in a par block. Since we are at the top level of a module each assignment statement will execute in parallel in its own infinite loop at clock rate[9].

Procedure and function calls can be recursive. This is a very powerful mechanism allowing very simple and concise code to be written. Recursion often offers solutions that cannot be written iteratively. Any target code is duplicated on each call.

We could instead write the above example recursively -

```
global function
del (value(in), int(n, 1)) {
    if (n == 0)
        return(in);
    static(d, type(in));
    [d <- in;]
    return(del(d, n-1));
}
```

Each call to the function delays zero or one clock cycles. If the count is non-zero it delays the input by one cycle then calls itself with the result and a count minus 1. The assignment again is within an un-named module but now it is a single statement. There will be n un-named modules created, each with one assignment.

When writing iterative code note that it can be convenient to use a value variable which is re-assigned on every iteration. The following function ORs the bits of its argument.

```
global function
orbits (value(v, "bits:")) {
    int(n, dimension(v));
    value(ret, "log", cast(getbit(v, 0), "log"));
    for (int(i, 1) ; i<n ; i++)
        ret := ret || cast(getbit(v, i), "log")
    return(ret);
}
```

[9]The code generator will eliminate the implied infinite loops and simply control the enables to the register flip-flops. How and when these are asserted depends on how execution in the FPGA is started. It can be started by an external signal or it can start spontaneously after configuration - see target-execution-start in the manual index. A static variable can have its 'continuous' attribute set to true in which case the enables will be tied high.

The value variable **ret** is initially the LSB of the input but each time around the loop it is re-assigned the OR of itself and the next successive bit from the input. Finally it is returned as the value of the function. Note that the function parameter has no data width - this is derived from the argument. We could make two changes to this example. First we could generalise it so that it would accept any type for the input. Secondly we could avoid the use of the library function **getbit()** by casting the input into an array of "log".

```
global function
orbits (value(v)) {
    int(n, size(v));
    value(vx,, forcecast(v, "["+n+"]log"));
    value(ret, "log", vx[0]);
    for (int(i, 1) ; i<n ; i++)
        ret := ret || vx[i]
    return(ret);
}
```

The library function **forcecast()** is a more extreme version of inbuilt function **cast()**. Where a value variable is given an initial value the type argument can be omitted since the type can be inferred from the initial value.

Module, procedure, function and class definitions may be nested, so a function or procedure may be defined within another function or procedure and hence can only be called by the parent function or procedure. By this means the scope and hence accessibility of functions and procedures can be limited and procedure and function identifiers can be re-used locally without ambiguity.

It may be apparent by now that modules, procedures, functions and classes are only called during immediate execution - they are not called within target code. There is no equivalent to a procedure or function call in the FPGA code. Indeed this would not make sense as it implies shared code and sequential access which defeats the purpose of using an FPGA for parallel execution. Procedures and functions are useful where the same code is needed multiple times, be it immediate code or target code, however the procedure and function calls only take place during immediate execution.

Module, procedures, functions and classes accept a "varargs" parameter which will match any number of arguments of any type or mode. This must follow normal parameters if used.

17 program structure and scope

There are a number of variable scopes -

- global scope
- file scope
- local scope
- block scope

Global scope is accessible anywhere. Variables are created in global scope if the variable identifier has "global." prepended when the variable creation inbuilt procedure is called. If the same variable identifier is used in more than one scope the global version can be explicitly selected by prepending "global.". Variables are never placed in global scope by default.

When the 3PL compiler begins to compile source code from the source file passed to it on the command line it creates a module to contain that code. That module is called a **file** module. A file module does not have an identifier since it is not explicitly called. Source code within that source file forms the code body of the file module and this is executed by the interpreter following compilation. Variables created immediately within the file module (i.e. not within nested blocks) by default are contained in the file module scope. If variables are created within code blocks nested within that file scope but their scope is required to be the file scope then their identifiers must have "file." prepended. Similarly if necessary a variable reference can be disambiguated by prepending "file.".

3PL allows other source files to be nested within the command line source file. There are two mechanisms for doing that.

A source file can be nested within another by the construction

```
source "file.3pl"
```

This simply inserts the text of the file at that point in the current file. This can be done to any depth. This is equivalent to #include in C. If instead the code

```
module "file.3pl"
```

is used the file is inserted as before however a new file module is created for which the new source file supplies the code body. again this can be done to any depth.

Any variables created immediately within the new file module are by default contained in the file scope for that module. Again this can be made explicit by prepending "file." to the variable identifier.

File scope is useful for hiding variables away from application code without the need to call a module for initialisation. For example a hardware interface is usually coded as a file module. The source file for this might be "gadget.3pl". By including this using the statement

```
module "gadget.3pl"
```

a file module will be created. Top level code within this file module will be interpreted at the point at which occurs in the enclosing source. This code will create variable, such as inputs, outputs, FPGA pin names etc. and by default these will be in the file module scope. The module will also define modules, procedures and functions which provide the interface to the gadget. These can access the variables in file scope but the callers to these cannot access the variables. Hiding variables in this way both protects the interface from incorrect usage and also avoids accidental conflicts with variable identifiers.

Within a module other than a file module, a procedure or a function variables created outside of nested code blocks are by default local to the module, procedure or function. They can be accessed anywhere within the module, procedure or function but not from any nested calls or from anywhere else.

Within a code block in braces any created variables are in the scope of the block. A problem arises where variables are created conditionally. The code

```
procedure
p (log(1)) {
    if (1)
        int(i);
}
```

does what one would expect - creates the integer **i** in the scope of the enclosing procedure. If this is changed to

```
procedure
p (log(1)) {
    if (1) {
        int(i);
        float(f);
    }
}
```

two variables are created, but they are in the scope of the **if** code block in braces, not in the scope of the procedure. To fix this the code should be changed to

```
procedure
p (log(1)) {
    if (1) {
        int(local.i);
        float(local.f);
    }
}
```

which creates the variables in the procedure scope.

Suppose one wishes to write a procedure which creates variables. The code

```
procedure
myvar (str(name), log(signed)) {
    if (signed)
        static(name->, "int:8");
    else
        static(name->, "uint:8");
}
```

will create a variable whose identifier is derived from the string argument by using the indirect operator. The problem is that the variable is created in the local scope of the procedure, not the scope in which it is called. To achieve the latter it can be changed to

```
procedure
myvar (str(name), log(signed)) {
    str(n, "calling." + name);
    if (signed)
        static(n->, "int:8");
    else
        static(n->, "uint:8");
}
```

or more simply

```
procedure
myvar (str(name), log(signed)) {
    if (signed)
        static(("calling." + name)->, "int:8");
    else
        static(("calling." + name)->, "uint:8");
}
```

or even more simply

```
procedure
myvar (str(name), log(signed)) {
    static(("calling." + name)->, signed ? "int:8" : "uint:8");
}
```

When the interpreter encounters a reference to a variable it first searches the current code block, then back out through nested code blocks until it reaches the local scope of the enclosing module, procedure or function. This behaviour is the same as exhibited by other common programming languages such as C or Java. If the variable has not been found the interpreter looks in the file (module) scope. Finally it looks in the global scope. If the variable identifier has "global.", "file.", "calling." or "local" prepended the interpreter explicitly expects to find the variable in the named scope.

When a non-inbuilt module, procedure or function is called, the interpreter searches from the current local scope back through the scopes of nested module, procedure and function **declarations** (NOT calls) until it finds a match. If that fails it looks in global scope. Thus, like variables, module, procedure and function identifiers can be used in different scopes as long as there is no conflict. A module, procedure or function definition occupies the scope of the module, procedure or function in which it is defined unless the keyword **global** precedes the **module**, **procedure** or **function** keyword, in which case it has global scope, i.e. it can be called from anywhere.

Nested declarations allow modules, procedures and functions to be limited in scope so that they can only be called locally. This also allows module, procedure and function identifiers to be re-used locally.

18 Selectvalue mode

Selectvalue mode creates busses using three-state buffers. A selectvalue variable can have a number of conditional assignments. The following is an example using this mode -

```
selectvalue(bus, "bits:8");
value(address, "uint:4");
value(write, "log");
static(data1, "int:5");
static(data2, "uint:7");

when ((address == 3) && write)
    bus |- cast(data1, type(bus));
when ((address == 7) && write)
    bus |- cast(data2, type(bus));
```

The assignment operator for a selectvalue is `|-` which is intended to look a little like a diagram of a connection to a bus. A conditional assignment to an output mode variable uses the same assignment operator because again a three-state drive is involved. When variable **address** is 3 and **write** is true, drive **data1** onto **bus**. When variable **address** is 7 and **write** is true, drive **data2** onto **bus**. The value of variable **bus** can be used in expressions the same way as other modes such as value or static. It is often the case that a bus must be shared by data of different types. This example illustrates this. The bus type has been made **bits** and variables **data1** and **data2** have been cast to that type. In both cases the variables have fewer bits than **bus** so **data2** will be zero padded to 8 bits and **data1** will be sign extended to 8 bits.

19 Priority mode

Priority mode creates a priority encoder. The type must be "[]log" where the array dimension is at least as large as the number of inputs (it may be larger and will be trimmed). The encoder is combinatorial and has the same number of outputs as inputs (or one extra, see below). If one or more inputs is true the output corresponding to the lowest subscript input which is true will be true, all other outputs being false. The following example arbitrates between two queue variables which are to be written to a common bus -

```
selectvalue(bus, "uint:8"); // bus
priority(arb, "[10]log");   // bus arbitration variable
queue({q1, q2}, "uint:8");   // bus sources

arb[0] := <q1; // q1 is available for reading
arb[1] := <q2; // q2 is available for reading

when (arb[0])
    bus |- q1;
when (arb[1])
    bus |- q2;
```

Since the lowest subscript applies to **q1** it will have priority over **q2** if both are available simultaneously. The priority variable is dimensioned 10 however the unused higher entries will be ignored. An additional output is available via the inbuilt function **prielse()**, e.g. **prielse(arb)**, which is true if none of the inputs is true.

20 More on types

Target variable types are -

bits:n	unspecified binary
uint:n	unsigned integer
int:n	signed integer
log	logical
fixed:m .n	fixed point
float:m en	floating point
	enumerated type
null	no data

Immediate types are -

bits	unspecified binary
uint	unsigned integer
int	signed integer
log	logical
str	string
float	floating point
->	pointer
type	type
list	a list
map	a map
file	a file
empty	take the type from an initialisation argument

In both target and immediate modes the types can be nested within arrays or structs. Immediate and target mode types cannot be mixed within a struct.

21 Inbuilt modules, procedures and functions

We have already seen the use of inbuilt procedures such as **static()** and **queue()**. There are very many such inbuilt procedures, even more inbuilt functions and a few inbuilt modules. The inbuilt functions include the usual numeric and string operations which are for immediate execution only.

22 Libraries

3PL libraries are source code files that are incorporated into the source stream by **source "..."** or **module "..."** statements. It is usual to incorporate a library called a *header file* as the first line of source code. This header file is specific to the FPGA platform and sets up FPGA package pins definitions and clock logic and provides interfaces to external devices on the platform. There are other libraries which are more general and contain useful modules, procedures and functions. Often these are already incorporated in the header file.

23 Petri nets

State machines are a convenient way of expressing a control algorithm. When implementing state machines in FPGAs it is usual to represent each state by a flip-flop state[10]. Exactly one state flip-flop should be set at any time. A state transition requires the current state flip-flop to be reset and another state flip-flop to be simultaneously set. A Petri net is a similar but more general scheme. The form of Petri net implemented in 3PL would be described as "a finite capacity Petri net with a capacity of 1" which is a less general form. Although it is a contradiction in terms one can think of this form of Petri net as a state machine implementation where more than one state flip-flop can be set at a time. Transitions can then involve some flip-flops being cleared and others set. A simple state machine can be described by a Petri net.

[10]The alternative is to represent the state by a counter, which is the usual way in software.

In a Petri net we have *places* rather than *states* and each place can have a token (set) or no token (cleared). A transition involves taking tokens from one or more places and inserting them in one or more other places. Expressions describing when transitions are to take place are called *predicates*.

The following is an example of the syntax -

```
petrinet {
    initial q1, q2;
    q1, q2 -> p3, p4 when (a && (b != 0)) {s <- queue1;};
    p3 ->> q1 {l1 <- queue1;};
    p4 -> q2, p3 when (!l2);
}
```

Here there are places **q1**, **q2**, **p3** and **p4**. Each of these is a static variable of type "log". If they have not already been created the Petri net construct creates them. These variables cannot be assigned by any code, only by the Petri net mechanism itself, however their values can be used in the program code outside of the Petri net in the same way that ordinary variables are used.

In the example places **q1** and **q2** initially have tokens. When the expression $a\&\&(b! = 0)$ (called the *predicate*) is true and **q1** and **q2** both contain tokens (are set), **q1** and **q2** will have their tokens removed (are cleared) and places **p3** and **p4** have tokens inserted (are set). The single right arrow indicates a weak transition, meaning that prior to the transition any tokens in the destination places are ignored. If a double right arrow is used this signifies a strong transition, meaning that the transition can only occur if the destination places do not have tokens.

In the example above the code block to the right of the first transition is executed when the transition occurs. It may contain only a single target statement (which can of course be a compound statement such as **seq{}**, **par{}** etc.). If a transition does not have a predicate then it occurs when the source places have tokens (and optionally destination places lack tokens). A transition code block may contain immediate statements but these would serve little purpose as they are only executed as the interpretter passes through the code during immediate execution.

To code a state machine as a Petri net the following apply -

- each transition has only one source and one destination place
- only one place is initialised
- transitions can be weak (it doesn't matter which are used, but weak transitions use fewer logic resources)